Notice how right rotation is always applied to left children, and left rotation to right children.

A rotation, in a binary search tree, is a transformation whose goal is inverting parent-child relation between two nodes of the tree, Y and X in figure 3.4: we want the child node to become the parent node and vice versa, but we can't just swap those two nodes, otherwise (in the general case where the keys of the two nodes are different) we would end up violating the ordering of keys.

Instead, what we need to do is removing the whole subtree rooted at the parent, replace it with the (smaller) subtree rooted at the child, and then find a way to plug back in the removed nodes in this new subtree.

How are we going to do that? As you can see in figure 3.4, first we need to distinguish two cases, depending on whether the child node is a left or a right child. The two cases are symmetrical, so we'll mainly focus on the former.

Listings 3.2 and 3.3 show the pseudocode for right and left rotations, explicating the details of the operations we described a few lines above; figure 3.5 illustrates the steps needed for right rotations, where the child node X, the pivot of the rotation, is the left child of its parent Y.

We need to remove Y from the tree, update Y's parent P (lines #3-#9), replacing Y with node X as its child (either left or right, see lines #6-#8); at this point, Y is disconnected from the tree, and with Y its whole right subtree.

Listing 3.2 Right Rotation

```
function rightRotate(treap, x)
                              #1
```